02.01.2026

Part 2: Testing Documentation for Smart Library Management System

1. Boundary Value Testing (BVT)

Method: `getRemainingBorrowCapacity(int memberId)` - Calculates remaining borrow capacity.

Purpose: Test values at input domain edges where errors are most likely. The boundary is maximum loans (N = 5).

Formula: `Math.max(0, maxLoans - activeBorrows)`

Test Case	Active Borrows	Expected	Rationale
Below (N-1)	4	1	One below limit
At (N)	5	0	At maximum
Above (N+1)	6	0	Math.max prevents negatives

Key Finding: Above boundary validates Math.max prevents negative returns when limits are exceeded.

2. Equivalence Class Testing

Method: `getFineCategory(String itemCode)` - Categorizes fines by overdue days.

Purpose: Partition inputs into classes where all values produce identical behavior, reducing redundant tests.

Class	Condition	Test Values	Expected Output
1. Invalid	null/empty/whitespace	null, "", " "	IllegalArgumentException

2. No Borrow	No record exists	-	IllegalStateException
3. Returned	Already returned	-	NO_FINE_RETURNED
4. On Time	overdueDays ≤ 0	-5, 0	NO_FINE_ON_TIME
5. Low Fine	1 ≤ overdueDays ≤ 5	1, 3, 5	LOW_FINE
6. Medium Fine	6 ≤ overdueDays ≤ 14	6, 10, 14	MEDIUM_FINE
7. High Fine	overdueDays > 14	15, 100	HIGH_FINE

Strategy: Test representative values (minimum, typical, maximum) from each class for complete coverage.

3. Code Coverage Testing

Method: `shouldSendReminder(String itemCode, int daysBeforeDue)` - Determines if reminder is needed.

i. Statement Coverage (100%): All executable lines tested including validation, range checks, database lookups, calculations, and returns.

ii. Branch Coverage (100%): All decisions tested TRUE/FALSE:

- `daysBeforeDue < 0`, `daysBeforeDue > 7`
- `record == null`, `record.isReturned()`
- `daysUntilDue == daysBeforeDue`, `daysUntilDue < 0`

iii. Condition Coverage (100%): Each atomic condition tested independently in compound expressions:

- `(daysBeforeDue < 0 || daysBeforeDue > 7)` - both parts TRUE/FALSE
- `(record == null || record.isReturned())` - both parts TRUE/FALSE

iv. MC/DC Coverage (100%): Proves each condition independently affects outcome. Example: `(daysBeforeDue < 0 || daysBeforeDue > 7)`

- Base: Both FALSE (value=3) → FALSE
- First TRUE (value=-2) → TRUE (proves first matters)
- Second TRUE (value=10) → TRUE (proves second matters)

Result: 100% coverage across all criteria ensures thorough testing of logic paths and edge cases.

Worked by: Megi Almadhi